# LAPORAN PRAKTIKUM
# PEMROGRAMAN MOBILE
# MODUL 2

## Android Layout With Compose

**Oleh:**

**Amandha Citra Mustika NIM. 2410817320004**

**PROGRAM STUDI TEKNOLOGI INFORMASI**
**FAKULTAS TEKNIK**
**UNIVERSITAS LAMBUNG MANGKURAT**
**APRIL 2026**

# LEMBAR PENGESAHAN
# LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
# MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Amandha Citra Mustika
NIM : 2410817320004

| | |
|---|---|
| Menyetujui,<br>Asisten Praktikum | Mengetahui,<br>Dosen Penanggung Jawab Praktikum |
| Muhammad Erza Raihan<br>NIM. 2310817210027 | Irham Maulani Abdul Gani, S.Kom., M.Kom.<br>NIP. 199710312025061009 |

# DAFTAR ISI

LEMBAR PENGESAHAN ..... 2
DAFTAR ISI ..... 3
DAFTAR GAMBAR ..... 4
DAFTAR TABEL ..... 5
SOAL 1 ..... 6
A. Source Code ..... 8
B. Output Program ..... 18
C. Pembahasan ..... 21
D. Tautan Git ..... 25

# DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi ..... 6
Gambar 2. Tampilan Pilihan Persentase Tip ..... 7
Gambar 3. Tampilan Aplikasi Setelah Dijalankan ..... 7
Gambar 4. Screenshot Hasil Jawaban Soal 1 XML ..... 18
Gambar 5. Screenshot Hasil Jawaban Soal 1 XML ..... 19
Gambar 6. Screenshot Hasil Jawaban Soal 1 XML ..... 19
Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose ..... 20
Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose ..... 20
Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose ..... 21

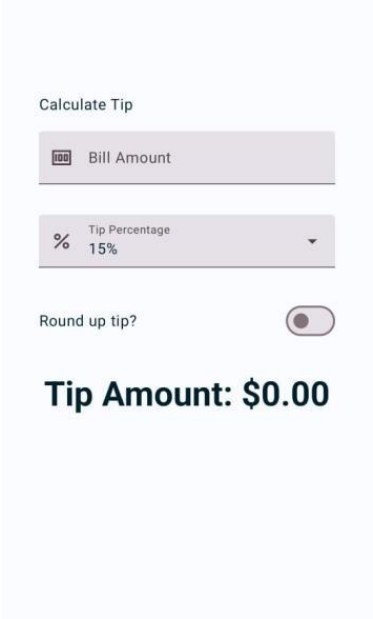
# DAFTAR TABEL

Table 1. Source Code MainActivity.kt XML Soal 1 ..... 8
Table 2. Source Code activity_main.xml Soal 1 ..... 11
Table 3. Source Code Jawaban Soal 1 XML ..... 13
Table 4. Source Code Jawaban Soal 1 Compose ..... 13

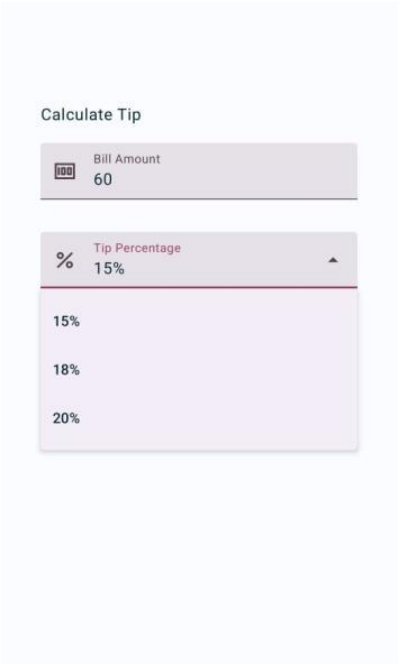
# SOAL 1

**Soal Praktikum:**

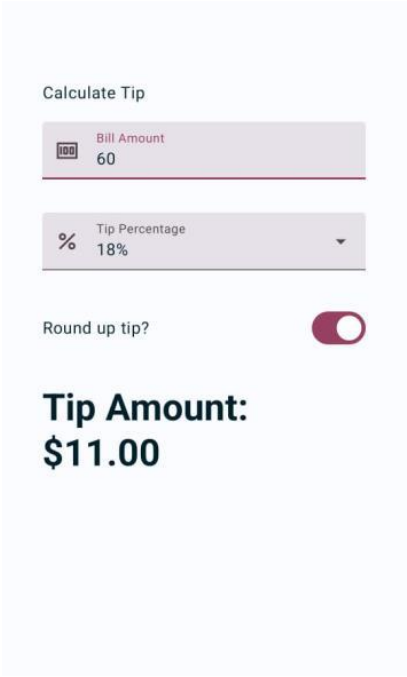
1. Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:
   a. Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
   b. Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
   c. Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
   d. Tampilan hasil: Aplikasi akan menampilkan jumlah tip yang harus dibayar secara langsung setelah pengguna memberikan input.
2. Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.




*Gambar 1. Tampilan Awal Aplikasi*




*Gambar 2. Tampilan Pilihan Persentase Tip*




*Gambar 3. Tampilan Aplikasi Setelah Dijalankan*

Additional:

1. Apa saja fitur yang menjadi daya jual pada Kotlin? (contoh: null safety, chain, dll)**
2. berikan 1 test case terkait penggunaan fitur unggulan yang ada pada kotlin (contoh: penggunaan null safety)

## A. Source Code

**MainActivity.kt XML**

*Table 1. Source Code MainActivity.kt XML Soal 1*

```
package com.example.calculator_xml

import android.os.Bundle
import android.text.Editable
import android.text.TextWatcher
import android.widget.ArrayAdapter
import android.widget.AutoCompleteTextView
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import com.google.android.material.materialswitch.MaterialSwitch
import com.google.android.material.textfield.TextInputEditText
import java.text.NumberFormat
import java.util.Locale
import kotlin.math.ceil

class MainActivity : AppCompatActivity() {

    private lateinit var etInputAmount: TextInputEditText
    private lateinit var autoCompleteTipPercentage: AutoCompleteTextView
    private lateinit var switchRoundUp: MaterialSwitch
```

```
    private lateinit var tvTipResult: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        etInputAmount = findViewById(R.id.etInputAmount)
        autoCompleteTipPercentage = findViewById(R.id.autoCompleteTipPercentage)
        switchRoundUp = findViewById(R.id.switchRoundUp)
        tvTipResult = findViewById(R.id.tvTipResult)

        val percentages = listOf("15%", "18%", "20%")
        val adapter = ArrayAdapter(this, android.R.layout.simple_list_item_1, percentages)
        autoCompleteTipPercentage.setAdapter(adapter)

        etInputAmount.addTextChangedListener(object : TextWatcher {
            override fun afterTextChanged(s: Editable?) { calculateTip() }
            override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int, after: Int) {}
            override fun onTextChanged(s: CharSequence?, start: Int, before: Int, count: Int) {}
        })
     autoCompleteTipPercentage.setOnItemClickListener { _, _, _, _ ->
            calculateTip()
        }

```

```
        switchRoundUp.setOnCheckedChangeListener { _, _ ->
            calculateTip()
        }
        calculateTip()
    }

    private fun calculateTip() {
        val amountStr = etInputAmount.text.toString()
        val amount = amountStr.toDoubleOrNull() ?: 0.0

        val percentageStr = autoCompleteTipPercentage.text.toString().replace("%", "")
        val percentage = percentageStr.toDoubleOrNull() ?: 15.0

        var tip = amount * (percentage / 100.0)

        if (switchRoundUp.isChecked) {
            tip = ceil(tip)
        }

        val formattedTip = NumberFormat.getCurrencyInstance(Locale.US).format(tip)
        tvTipResult.text = getString(R.string.tip_amount, formattedTip)
    }
}
```

**activity_main.xml**

*Table 2. Source Code activity_main.xml Soal 1*

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView
xmlns:android="http://schemas.android.com/apk/res/androi
d"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:paddingHorizontal="32dp"
        android:paddingVertical="40dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/calculate_tip"
            android:textSize="24sp"
            android:layout_marginBottom="16dp"/>


<com.google.android.material.textfield.TextInputLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="@string/bill_amount"
style="@style/Widget.MaterialComponents.TextInputLayout.
FilledBox"
            android:layout_marginBottom="16dp">
<com.google.android.material.textfield.TextInputEditText
                android:id="@+id/etInputAmount"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:inputType="numberDecimal" />

</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
style="@style/Widget.MaterialComponents.TextInputLayout.
FilledBox.ExposedDropdownMenu"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
```

```
            android:hint="@string/tip_percentage"
            android:layout_marginBottom="16dp">

            <AutoCompleteTextView
android:id="@+id/autoCompleteTipPercentage"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:inputType="none"
                android:text="15%" />

</com.google.android.material.textfield.TextInputLayout>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal"
            android:gravity="center_vertical"
            android:layout_marginBottom="20dp">

            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="@string/round_up_tip"
                android:textSize="16sp"/>

<com.google.android.material.materialswitch.MaterialSwit
ch
                android:id="@+id/switchRoundUp"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content" />
        </LinearLayout>

        <TextView
            android:id="@+id/tvTipResult"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="@string/tip_amount"
            android:textSize="30sp"
            android:textStyle="bold"
            android:gravity="center" />
    </LinearLayout>
</ScrollView>
```

**strings.xml**

*Table 3. Source Code Jawaban Soal 1 XML*

```
<resources>
    <string name="app_name">Calculator_XML</string>
    <string name="calculate_tip">Calculate Tip</string>
    <string name="bill_amount">Bill Amount</string>
    <string name="tip_percentage">Tip Percentage</string>
    <string name="round_up_tip">Round up tip?</string>
    <string name="tip_amount">Tip Amount: %s</string>

</resources>
```

**MainActivity.kt Compose**

*Table 4. Source Code Jawaban Soal 1 Compose*

```
package com.example.tipcalculator

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.tipcalculator.ui.theme.TipCalculatorTheme
import java.text.NumberFormat
import androidx.compose.foundation.text.KeyboardOptions
```

```
17 import androidx.compose.foundation.verticalScroll
18 import
   androidx.compose.runtime.saveable.rememberSaveable
19 import androidx.compose.ui.Alignment
20 import androidx.compose.ui.text.font.FontWeight
21 import androidx.compose.ui.text.input.KeyboardType
22 import androidx.compose.ui.unit.sp
23 import java.util.Locale
24 import kotlin.math.ceil
25
26 class MainActivity : ComponentActivity() {
27     override fun onCreate(savedInstanceState: Bundle?) {
28         super.onCreate(savedInstanceState)
29         enableEdgeToEdge()
30         setContent {
31             TipCalculatorTheme {
32                 Scaffold(modifier =
   Modifier.fillMaxSize()) { innerPadding ->
33                     TipCalculatorApp(
34                         modifier = Modifier
35                             .padding(innerPadding)
36                             .padding(horizontal = 32.dp,
   vertical = 40.dp)
37                     )
38                 }
39             }
40         }
41     }
42 }
43
44 @OptIn(ExperimentalMaterial3Api::class)
```

```
@Composable
@Preview(showBackground = true)
fun TipCalculatorApp(modifier: Modifier = Modifier) {
    var inputAmount by rememberSaveable { mutableStateOf("") }
    var expanded by rememberSaveable { mutableStateOf(false) }
    var selectedPercentage by rememberSaveable { mutableIntStateOf(15) }
    var roundUp by rememberSaveable { mutableStateOf(false) }

    val amount = inputAmount.toDoubleOrNull() ?: 0.0
    var tip = amount * (selectedPercentage / 100.0)

    if (roundUp) {
        tip = ceil(tip)
    }

    val formattedTip = NumberFormat.getCurrencyInstance(Locale.US).format(tip)
    val percentages = listOf(15, 18, 20)

    Column(
        modifier = modifier
            .fillMaxSize()
            .verticalScroll(rememberScrollState()),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Text(
```

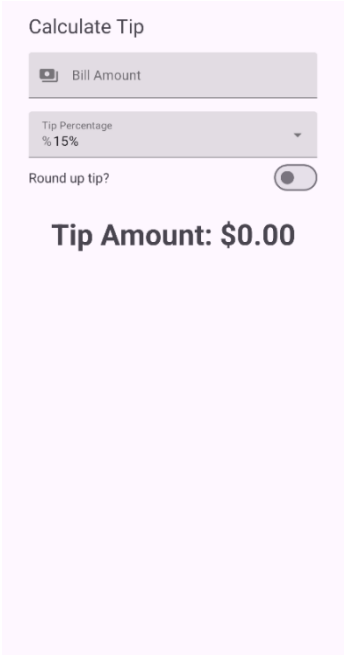
```
            text = "Calculate Tip",
            fontSize = 24.sp,
            modifier = Modifier.align(Alignment.Start)
        )

        TextField(
            value = inputAmount,
            onValueChange = { inputAmount = it },
            label = { Text("Bill Amount") },
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
            modifier = Modifier.fillMaxWidth()
        )

        ExposedDropdownMenuBox(
            expanded = expanded,
            onExpandedChange = { expanded = !expanded }
        ) {
            OutlinedTextField(
                value = "$selectedPercentage%",
                onValueChange = {},
                readOnly = true,
                label = { Text("Tip Percentage") },
                leadingIcon = { Text(text = "%", fontSize = 18.sp) },
                trailingIcon = {
ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded)
                },
                modifier = Modifier
```

```
.menuAnchor(MenuAnchorType.PrimaryNotEditable)
                    .fillMaxWidth()
            )
            ExposedDropdownMenu(
                expanded = expanded,
                onDismissRequest = { expanded = false }
            ) {
                percentages.forEach { percentage ->
                    DropdownMenuItem(
                        text = { Text("$percentage%") },
                        onClick = {
                            selectedPercentage = percentage
                            expanded = false
                        }
                    )
                }
            }
        }

        Row(
            modifier = Modifier.fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Text("Round up tip?")
            Switch(
                checked = roundUp,
                onCheckedChange = { roundUp = it }
```

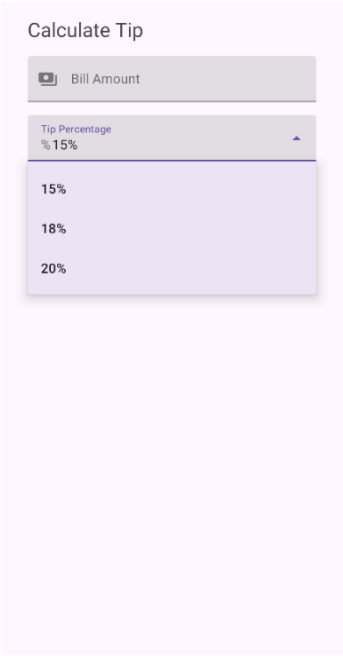
```
            )
        }
        Spacer(modifier = Modifier.height(20.dp))

        Text(
            text = "Tip Amount: $formattedTip",
            fontSize = 35.sp,
            fontWeight = FontWeight.Bold,
            modifier =
Modifier.align(Alignment.CenterHorizontally)
        )
    }
}
```

## B. Output Program

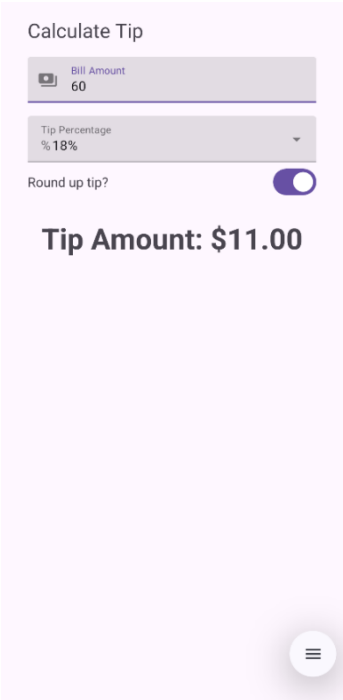
**XML:**




*Gambar 4. Screenshot Hasil Jawaban Soal 1 XML*

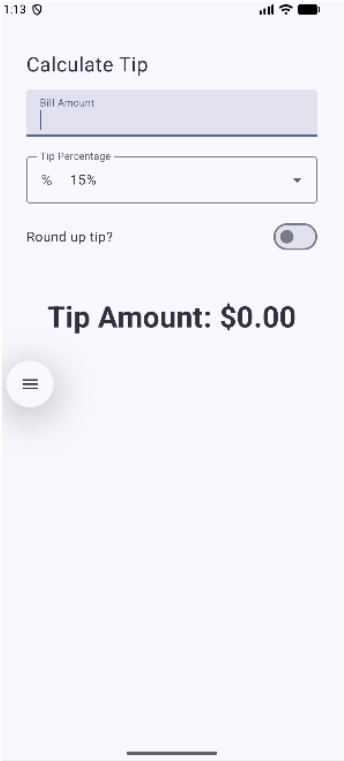



*Gambar 5. Screenshot Hasil Jawaban Soal 1 XML*



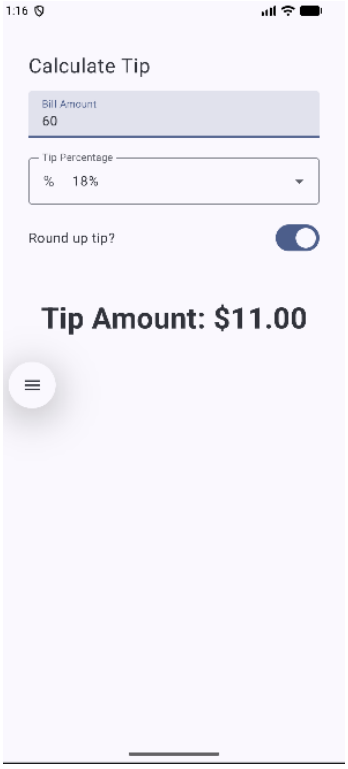

*Gambar 6. Screenshot Hasil Jawaban Soal 1 XML*

**Compose**




*Gambar 7. Screenshot Hasil Jawaban Soal 1 Compose*

*Gambar 8. Screenshot Hasil Jawaban Soal 1 Compose*




*Gambar 9. Screenshot Hasil Jawaban Soal 1 Compose*

## C. Pembahasan

### MainActivity.kt XML

Pada file MainActivity.kt, logika aplikasi mulai berjalan di dalam fungsi ‘onCreate()’. Fungsi ‘setContentView(R.layout.activity_main)’ dipanggil untuk menghubungkan class Kotlin ini dengan desain XML yang telah dibuat. Lalu, dilakukan proses inisialisasi variabel menggunakan ‘findViewById’ untuk menghubungkan komponen UI seperti kolom input tagihan (etInputAmount), menu dropdown persentase (autoCompleteTipPercentage), switch pembulatan (switchRoundUp), dan teks hasil (tvTipResult) dengan variabel di dalam kode Kotlin. Setelah inisialisasi, program langsung menyiapkan opsi menu dropdown dan interaksi pengguna di dalam fungsi ‘onCreate()’ tersebut.

Untuk bagian menu dropdown, opsi "15%", "18%", dan "20%" dimasukkan ke dalam daftar menggunakan ArrayAdapter, lalu diatur ke dalam komponen ‘autoCompleteTipPercentage’. Untuk menangkap interaksi, program menambahkan TextWatcher pada kolom input agar setiap ketikan memicu aksi ‘afterTextChanged’. Selain

itu, program juga menambahkan ‘setOnItemClickListener’ pada dropdown dan ‘setOnCheckedChangeListener’ pada tombol switch. Ketiga listener ini berfungsi untuk memastikan bahwa setiap perubahan, baik itu ketikan angka, pemilihan persentase, maupun geseran tombol, akan langsung memanggil fungsi ‘calculateTip()’ secara otomatis.

Aksi utama perhitungannya terjadi di dalam fungsi calculateTip(). Program mengambil teks tagihan yang diketik pengguna dan mengubahnya menjadi tipe desimal ganda (Double), serta mengambil teks persentase dengan menghilangkan simbol "%" agar bisa dihitung. Setelah perhitungan perkalian dasar didapatkan, program menggunakan percabangan if untuk mengecek status ‘switchRoundUp’ jika sakelar menyala, nilai tip akan dibulatkan ke bilangan bulat terdekat ke atas menggunakan fungsi ceil(). Terakhir, program menggunakan ‘NumberFormat.getCurrencyInstance’ untuk memformat nilai akhir menjadi format mata uang dolar, lalu memperbarui tvTipResult pada layar untuk menampilkan hasilnya.

**activity_main.xml**

Pada file activity_main.xml, antarmuka pengguna (UI) dibangun menggunakan komponen ‘ScrollView’ sebagai wadah paling luar agar layar dapat digulir. Di dalamnya, terdapat LinearLayout utama dengan orientasi vertikal yang menyusun semua elemen secara berurutan dari atas ke bawah, dilengkapi dengan ‘padding horizontal 32dp dan vertikal 40dp’ agar tampilan memiliki ruang yang rapi. Elemen pertama yang dimuat di dalam wadah ini adalah sebuah TextView yang berfungsi sebagai judul halaman bertuliskan "Calculate Tip" dengan ukuran huruf 24sp.

Selanjutnya, tata letak menampilkan kolom input untuk nominal tagihan yang dibungkus menggunakan ‘TextInputLayout’ dengan gaya bawaan FilledBox. Di dalam pembungkus ini, disematkan ‘TextInputEditText’ ber-ID ‘etInputAmount’ yang secara spesifik diatur agar hanya menerima ketikan angka desimal. Tepat di bawahnya, terdapat menu dropdown yang juga dibungkus oleh ‘TextInputLayout’ namun diberikan gaya tambahan ‘ExposedDropdownMenu’ agar berfungsi sebagai menu tarik-turun. Di dalam bungkus tersebut, diletakkan ‘AutoCompleteTextView’ ber-ID ‘autoCompleteTipPercentage’ yang menampilkan teks bawaan "15%" dan input tipenya dinonaktifkan (none) agar pengguna tidak bisa mengetik manual.

Untuk fitur pembulatan angka, program menyisipkan wadah LinearLayout baru yang berorientasi horizontal agar elemen di dalamnya tersusun menyamping dan sejajar di tengah (center_vertical). Di dalam wadah penyambung ini, sebuah ‘TextView’ yang berisi teks "Round up tip?" dibentangkan menggunakan layout_weight="1" agar menyeimbangkan dan mendorong komponen ‘MaterialSwitch’ ber-ID ‘switchRoundUp’ ke sisi kanan layar. Sebagai penutup antarmuka, tata letak diakhiri dengan sebuah ‘TextView’ ber-ID ‘tvTipResult’. Komponen ini diatur agar posisinya berada di tengah dengan ukuran huruf 30sp dan dicetak tebal untuk menonjolkan fungsinya sebagai penampil hasil akhir.

**MainActivity.kt Compose**

Pada file MainActivity.kt versi Compose, aplikasi dijalankan melalui fungsi setContent yang memuat antarmuka di dalam blok Scaffold dan memanggil composable utama bernama ‘TipCalculatorApp()’. Di dalam fungsi utama ini, program mendeklarasikan beberapa variabel state menggunakan ‘rememberSaveable’ agar data tidak hilang saat layar diputar. State tersebut meliputi ‘inputAmount’ untuk teks tagihan, expanded untuk status buka-tutup menu dropdown, ‘selectedPercentage’ untuk nilai tip dalam bentuk integer, dan roundUp untuk status sakelar pembulatan. Program langsung mengeksekusi logika perhitungan secara reaktif dengan mengonversi ‘inputAmount’ menjadi angka, menghitung nilai tip, dan menerapkan fungsi ‘ceil()’ jika state ‘roundUp’ bernilai true. Antarmuka utama dibungkus oleh komponen Column yang sifatnya bisa digulir berkat ‘verticalScroll’ serta diberikan jarak antar komponen sebesar 16dp.

Untuk bagian menu persentase tip, tata letak menggunakan ‘ExposedDropdownMenuBox’. Di dalamnya, terdapat sebuah ‘OutlinedTextField’ yang diatur readOnly untuk berfungsi sebagai area klik. Saat area ini ditekan, nilai state expanded akan berbalik dan memunculkan ‘ExposedDropdownMenu’. Menu ini menggunakan fungsi forEach untuk menampilkan deretan angka dari list persentase ke dalam bentuk ‘DropdownMenuItem’. Saat salah satu opsi dipilih, state ‘selectedPercentage’ akan diperbarui dan menu akan kembali ditutup.

Pada bagian fitur pembulatan, program menggunakan komponen Row dengan pengaturan Arrangement.SpaceBetween agar teks "Round up tip?" berada di sisi kiri penuh dan komponen Switch terdorong ke sisi kanan. Komponen Switch ini dihubungkan secara

dua arah dengan state roundUp sehingga setiap gesekan langsung memperbarui state dan memicu perhitungan ulang pada antarmuka. Sebagai penutup, tepat di bawah Spacer yang memberikan jarak ruang, terdapat komponen Text berukuran 35sp dengan format tebal. Teks ini secara khusus bertugas menampilkan nilai ‘formattedTip’ yang sebelumnya sudah diubah ke format mata uang menggunakan pustaka ‘NumberFormat’.

- Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

1. Implementasi Menggunakan XML Layout

XML merupakan pendekatan konvensional yang memisahkan antara desain struktur visual dan logika pemrograman.

- **Kelebihan:** Keunggulan utama dari XML adalah ekosistemnya yang telah sangat matang. Dengan usia pengembangan lebih dari satu dekade, ketersediaan dokumentasi, pustaka pihak ketiga, dan solusi atas berbagai galat sangat komprehensif. XML juga difasilitasi oleh fitur Layout Editor di Android Studio yang memungkinkan perancangan antarmuka secara visual. Selain itu, pemisahan struktur yang eksplisit antara antarmuka berupa berkas XML dan logika berupa berkas Kotlin atau Java memudahkan pembagian tugas saat bekerja dalam tim.
- **Kekurangan:** Dari segi pemeliharaan, sintaks XML cenderung panjang dan repetitif, sehingga menyulitkan manajemen kode pada antarmuka yang kompleks. Pendekatan ini juga mewajibkan pengembang untuk mengelola pembaruan status data ke antarmuka secara manual melalui fungsi pengamat, yang rentan memicu kecacatan sinkronisasi. Selain itu, sistem View pada XML memerlukan proses inflate, yaitu tahapan menerjemahkan kode XML menjadi objek antarmuka saat aplikasi dijalankan, yang memberikan beban komputasi tambahan pada performa aplikasi.

2. Implementasi Menggunakan Jetpack Compose

Jetpack Compose adalah perangkat kerja antarmuka modern dari Google yang menyatukan deklarasi antarmuka dan logika sistem di dalam satu bahasa pemrograman, yaitu Kotlin.

- **Kelebihan:** Keuntungan paling signifikan dari Compose adalah efisiensi penulisan kode. Antarmuka dapat dibangun dengan baris kode yang jauh lebih ringkas dibandingkan XML. Menggunakan konsep manajemen status, Compose bersifat reaktif. Sistem akan secara otomatis menggambar ulang komponen antarmuka yang terdampak saat terjadi perubahan data melalui proses rekomposisi, sehingga meminimalisasi kode pembaruan manual. Karena direpresentasikan sebagai fungsi murni, setiap elemen antarmuka memiliki tingkat modularitas dan penggunaan ulang yang sangat tinggi, serta jauh lebih mudah untuk diuji secara independen.
- **Kekurangan:** Tantangan utama dalam mengadopsi Jetpack Compose adalah kurva pembelajarannya yang tinggi. Pengembang harus mengubah pola pikir dari gaya pemrograman imperatif ke deklaratif, serta memahami konsep arsitektur modern seperti pengangkatan status dan manajemen memori. Mengingat teknologinya yang relatif lebih baru, ekosistemnya masih dalam tahap perkembangan sehingga beberapa pustaka lama belum didukung secara optimal. Proses pelacakan galat juga menjadi lebih kompleks karena jejak rekaman sistem pada saat rekomposisi cukup rumit dipetakan. Terakhir, implementasinya dibatasi secara ketat pada proyek dengan standar spesifikasi batas bawah API versi 21.

## D. Tautan Git

https://github.com/zvyaa/MODUL-MOBILE

Additional:

1. Fitur yang menjadi daya jual Kotlin:
   - **Null Safety:** Pada Java, *error* NullPointerException (NPE) adalah salah satu kendala terbesar yang sering menyebabkan aplikasi berhenti mendadak (*force close*). Kotlin menyelesaikan masalah ini secara mendasar dengan membedakan tipe data yang boleh bernilai *null* (menggunakan akhiran ?) dan yang tidak boleh. Pendekatan ini memaksa pengembang untuk menangani kemungkinan data kosong sejak tahap kompilasi kode, bukan saat aplikasi sudah berjalan, sehingga menjamin stabilitas aplikasi secara signifikan (Griffiths & Griffiths, 2019).

- **Data Classes:** Pembuatan model kelas untuk menampung data (POJO) menjadi sangat efisien. Alih-alih menulis puluhan baris *boilerplate code* untuk fungsi *getter*, *setter*, equals(), hashCode(), dan toString(), kita hanya perlu menggunakan deklarasi satu baris dengan awalan data class. Kotlin akan membangun sisa logika tersebut secara otomatis di belakang layar.
- **Smart Casts:** Fitur ini menunjukkan kecerdasan dari *compiler* Kotlin. Jika program sudah memvalidasi tipe suatu variabel menggunakan operator kondisional is, *compiler* akan mengingat status tersebut. Kita tidak perlu lagi menulis ulang sintaks *casting* (konversi tipe data) secara manual di dalam blok eksekusi, sehingga kode menjadi jauh lebih rapi.
- **Extension Functions:** Kotlin memberikan keleluasaan untuk menyuntikkan fungsi atau perilaku baru ke dalam sebuah *class* yang sudah paten (termasuk *library* pihak ketiga) tanpa harus repot menggunakan teknik pewarisan (*inheritance*). Kode tambahan ini nantinya bisa dipanggil seolah-olah ia adalah fungsi bawaan (*built-in*) dari *class* itu sendiri, membuat alur pemrograman terasa lebih natural dan bersih (Skeen & Greenhalgh, 2018).

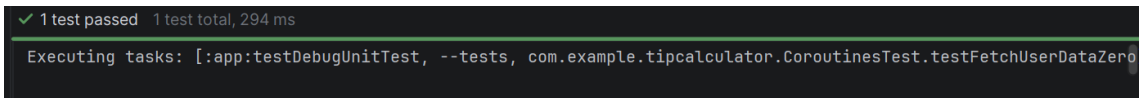
2. Test case untuk fitur Null Safety menggunakan operator Safe Call (?.) dan Elvis Operator (?:).

   Fungsi suspend fetchUserData mensimulasikan pengambilan data pengguna dari server. Pengujian memastikan fungsi mengembalikan data yang benar setelah operasi asinkron selesai.

```
 1 package com.example.tipcalculator
 2
 3 import kotlinx.coroutines.delay
 4 import kotlinx.coroutines.test.runTest
 5 import org.junit.Assert.assertEquals
 6 import org.junit.Test
 7
 8 class CoroutinesTest {
 9
10     private suspend fun fetchUserData(userId: Int):
11 String {
12         delay(1000)
13         return if (userId > 0) "Data pengguna dengan ID:
```

```
$userId" else "ID tidak valid"
    }

    @Test
    fun testFetchUserDataValid() = runTest {
        val result = fetchUserData(5)
        assertEquals("Data pengguna dengan ID: 5",
result)
    }

    @Test
    fun testFetchUserDataInvalid() = runTest {
        val result = fetchUserData(-1)
        assertEquals("ID tidak valid", result)
    }

    @Test
    fun testFetchUserDataZero() = runTest {
        val result = fetchUserData(0)
        assertEquals("ID tidak valid", result)
    }
}
```

Kode ini menguji fitur Coroutines Kotlin menggunakan fungsi suspend fetchUserData() yang mensimulasikan pengambilan data dari server dengan delay(1000) selama 1 detik. Penggunaan runTest dari library kotlinx-coroutines-test memungkinkan delay tersebut dilewati secara virtual sehingga test tetap berjalan cepat tanpa benar-benar menunggu 1 detik. Fungsi mengembalikan pesan berbeda tergantung nilai userId - jika lebih dari 0 maka data dikembalikan, jika tidak maka dikembalikan pesan "ID tidak valid".




Analisis Hasil:

testFetchUserDataValid menguji input userId = 5 yang merupakan angka positif valid, sehingga fungsi mengembalikan "Data pengguna dengan ID: 5" sesuai ekspektasi dan test dinyatakan Pass. testFetchUserDataInvalid menguji input userId = -1 yang merupakan angka negatif, fungsi mengembalikan "ID tidak valid" dan test Pass.

testFetchUserDataZero menguji input userId = 0 yang tidak memenuhi kondisi userId > 0, sehingga fungsi juga mengembalikan "ID tidak valid" dan test Pass. Ketiga test berhasil membuktikan bahwa Coroutines Kotlin mampu menjalankan operasi asinkron secara efisien tanpa memblokir thread utama, sekaligus memastikan logika fungsi berjalan benar untuk setiap kondisi input.

# DAFTAR PUSTAKA

Griffiths, D., & Griffiths, D. (2019). *Head First Kotlin: A Brain-Friendly Guide*. O'Reilly Media.

Skeen, J., & Greenhalgh, D. (2018). *Kotlin Programming: The Big Nerd Ranch Guide*. Big Nerd Ranch Guides.